

Scope fuction

Final memory table

Function Can Modify? Can Read? Returns Called As

with  Yes  Yes Last expression with(obj)

run  Yes  Yes Last expression obj.run

Step 2: Explain all five

Function Object inside lambda Returns Primary use

let it Lambda result Transform data or safely work with nullable objects

run this Lambda result Execute operations on an object and return a computed result

with this Lambda result Perform multiple operations on an existing object without repeating its name

apply this Same object Configure or initialize an object

also it Same object Perform side effects (logging, debugging, validation) while returning the object

Table Example

Function	Object inside lambda	returns	Primary use
let	it	lambda object	Transform data or safely work with nullable objects
run	this	lambda object	Execute operations on an object and return a computed result
with	this	lambda object	Perform multiple operations on an existing object without repeating its name
apply	this	same object	Configure or initialize an object
also	it	same object	Perform side effects (logging, debugging, validation) while returning the object

let

let uses it and returns the lambda result. It's commonly used for null safety (?.let) and transforming one object into another.

run

run uses this and returns the lambda result. It's useful when you want to execute multiple operations on an object and return a computed value. It's an extension function.

with

with is similar to run, but it's a regular function instead of an extension function. You pass the object as an argument and perform multiple operations on it.

apply

apply uses this and returns the object itself. It's mainly used for object configuration and initialization.

also

also uses it and returns the object itself. It's mainly used for side effects like logging, debugging, validation, or analytics without breaking a method chain.

sealed class

sealed classes are a special type of class that allow you to represent a restricted hierarchy of classes, where all possible subclasses are known at compile-time. This makes them ideal for modeling restricted data types or state hierarchies.

Unlike regular classes, sealed classes enforce a strict set of subclasses, allowing the compiler to ensure all cases are handled when performing when expressions, making code safer and more expressive.

LaunchedEffect is used for running suspend functions (coroutines) when a particular key changes. It runs its block of code when the Composable enters the composition and again when its keys change. It cancels any previous coroutine if the key changes.

Use Case:

Fetching data from a network or database when a particular state changes.

Showing a one-time toast message when the UI is first composed.

<https://proandroiddev.com/jetpack-compose-side-effects-iii-rememberupdatedstate-c8df7b90a01d>

<https://proandroiddev.com/jetpack-compose-side-effects-launchedeffect-59d2330d7834>

<https://proandroiddev.com/jetpack-compose-side-effects-ii-remembercoroutinescope-76104d7ff09>

rememberUpdatedState is used when we want to keep an updated reference to a variable in a long-running side-effect without having the side-effect to restart on recomposition

rememberCoroutineScope: When you need a coroutine to start based on a user action, such as a button click, rememberCoroutineScope is useful. It provides a scope tied to the composable's lifecycle,

ensuring the coroutine cancels if the composable leaves the composition.

produceState is used to convert external state, such as data from a network or database, into Compose state. It launches a coroutine that updates the state as necessary. This is particularly useful for managing state that is derived from external sources, such as fetching data from a remote API or database and then feeding that data into your composable's state.

1. launchIn()

Normally, to collect a Flow, you write:

```
viewModelScope.launch {  
    flow.collect {  
        // Handle data  
    }  
}
```

You can also write:

```
flow  
    .onEach {  
        // Handle data  
    }  
    .launchIn(viewModelScope)
```

This is exactly equivalent to:

```
viewModelScope.launch {  
    flow.collect()  
}
```

launchIn() starts collecting a Flow inside the provided CoroutineScope and returns a Job.

2. stateIn()

Problem

Suppose:

```
val userFlow = repository.getUsers()
```

This is a cold Flow.

Every time someone collects it:

Collector 1

↓

API Call

Collector 2

↓

API Call Again

Not good.

What does stateIn() do?

It converts:

Flow

into:

StateFlow

which:

Stores the latest value.

Shares it with all collectors.

Can have an initial value.

Example:

```
val users = repository.getUsers()  
    .stateIn(  
        scope = viewModelScope,
```

```
started = SharingStarted.WhileSubscribed(),  
initialValue = emptyList()  
)
```

Now:

API Call

↓

StateFlow

↓

Collector 1

Collector 2

Collector 3

Everyone gets the same latest value.

Because UI usually needs:

Current state of data.

Examples:

Loading

Success

Error

User details

Cart items

Theme mode

These are STATES.

So:

Flow

↓

stateIn()

↓

StateFlow

↓

UI observes state.

stateIn() converts a Flow into a StateFlow that always holds the latest value and is suitable for representing UI state.

3. shareIn()

This one is slightly different.

Problem

Suppose:

Flow

has:

Collector 1

↓

API Call

Collector 2

↓

API Call Again

Collector 3

↓

API Call Again

You don't want multiple executions.

So:

shareIn()

shares the Flow among multiple collectors.

Example

Without shareIn():

Flow

↓

Collector1

API Call

Collector2

API Call

Collector3

API Call

With:

shareIn()

API Call

↓

SharedFlow

↓

Collector1

Collector2

Collector3

Only ONE producer exists.

Why not use stateIn()?

Because shareIn():

Doesn't require an initial value.

Doesn't maintain a state.

Is used for sharing emissions.

Think:

Events

Examples:

Socket updates

Stock prices

Live cricket score

Chat messages

Location updates

You want:

One producer

↓

Many collectors

without keeping state.

Difference between `stateIn()` and `shareIn()`

`stateIn()`

Flow

↓

StateFlow

↓

Keeps latest value

↓

Requires initial value

Example:

User profile

Theme

Cart count

Login state

`shareIn()`

Flow

↓

SharedFlow

↓

Shares emissions



No initial value required

Example:

WebSocket messages

Chat messages

Location updates

Live events

A simple way to remember it is:

standard → Always create a new activity.

singleTop → Reuse only if it's on top.

singleTask → Reuse wherever it exists in the task (remove everything above it).

singleInstance → Reuse it, and it must live alone in its own task.

Interview Summary

Flag Purpose

NEW_TASK Creates or brings an existing task to the foreground

CLEAR_TOP Removes all activities above the target activity

SINGLE_TOP Reuses the activity if it is already on top

CLEAR_TASK Clears the entire back stack

NO_HISTORY Activity is not kept in the back stack

REORDER_TO_FRONT Moves an existing activity to the top without destroying others

MULTIPLE_TASK Forces creation of a separate task

"ContentResolver is an abstract class provided by the Android framework. We don't instantiate it ourselves; Android provides an instance through Context.contentResolver, which we use to interact with Content Providers."

A Fragment is a reusable and modular portion of an Activity's UI and behavior that has its own lifecycle and must be hosted inside an Activity.

A data class is a class that is specifically designed to hold data. The Kotlin compiler automatically generates useful methods such as equals(), hashCode(), toString(), copy(), and componentN() functions based on the properties declared in its primary constructor.

Data classes are typically used to create immutable objects that primarily hold data without requiring additional boilerplate code.

A companion object is a special object associated with a class that allows you to define properties and functions directly within the class, similar to static members in Java. However, unlike Java's static keyword, companion objects are more flexible and can be used as a singleton object that can hold methods and properties common to all instances of the class.